

Documento de Requisitos No Funcionales y Restricciones Técnicas

1. Introducción

El presente documento técnico detalla los Requisitos No Funcionales (RNF) y las restricciones técnicas inherentes al desarrollo de una plataforma de comercio electrónico de periféricos de PC, diseñada bajo una arquitectura de microservicios. A diferencia de los requisitos funcionales, que describen qué debe hacer el sistema, los RNF especifican cómo debe comportarse el sistema y las cualidades que debe poseer. Las restricciones técnicas, por su parte, son limitaciones impuestas por la arquitectura elegida o el entorno de desarrollo. Ambos aspectos son cruciales para el éxito del proyecto, ya que impactan directamente en la calidad, la viabilidad y la sostenibilidad a largo plazo de la solución.

Este documento está dirigido a arquitectos de software, ingenieros de desarrollo, especialistas en operaciones (DevOps) y stakeholders técnicos, con el fin de proporcionar una comprensión profunda de los desafíos y consideraciones que deben abordarse durante el ciclo de vida del proyecto.

2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales son atributos de calidad que el sistema debe exhibir. Su cumplimiento es tan crítico como el de los requisitos funcionales para la satisfacción del usuario y la eficiencia operativa.

2.1. Rendimiento

El rendimiento del sistema se refiere a la capacidad de respuesta, la velocidad y la eficiencia bajo diversas cargas de trabajo.

- RNF-PERF-001:** El tiempo de respuesta para las operaciones de búsqueda y filtrado de productos no debe exceder 1 segundo para el 95% de las solicitudes bajo una

carga de 1000 usuarios concurrentes.

- **RNF-PERF-002:** El tiempo de carga de la página principal y las páginas de listado de productos no debe exceder los 3 segundos para el 90% de los usuarios, incluso con una conexión de red promedio (ej. 5 Mbps).
- **RNF-PERF-003:** El tiempo de procesamiento de una transacción de pago completa (desde la confirmación del pedido hasta la notificación de éxito) no debe exceder los 5 segundos.
- **RNF-PERF-004:** El sistema debe ser capaz de procesar al menos 50 transacciones por segundo durante picos de demanda.

2.2. Mantenibilidad

La mantenibilidad se refiere a la facilidad con la que el sistema puede ser modificado, corregido o adaptado a nuevos requisitos.

- **RNF-MANT-001:** El código fuente de cada microservicio debe estar documentado siguiendo estándares de codificación (ej. Clean Code, SOLID principles) y convenciones de nomenclatura.
- **RNF-MANT-002:** Cada microservicio debe tener una cobertura de pruebas unitarias y de integración mínima del 80%.
- **RNF-MANT-003:** La arquitectura debe permitir el despliegue independiente de cada microservicio sin afectar la operación de otros servicios.
- **RNF-MANT-004:** El sistema debe contar con un sistema de logging y monitoreo centralizado que facilite la identificación y resolución de problemas en un tiempo máximo de 30 minutos.

2.3. Portabilidad

La portabilidad se refiere a la facilidad con la que el sistema puede ser transferido de un entorno operativo a otro.

- **RNF-PORT-001:** La plataforma debe ser desplegable en diferentes entornos de nube (ej. AWS, Google Cloud, Azure) utilizando tecnologías de contenedores (ej. Docker, Kubernetes).
- **RNF-PORT-002:** Los microservicios deben ser desarrollados utilizando lenguajes de programación y frameworks multiplataforma (ej. Java con Spring Boot, Node.js con

Express, Python con FastAPI).

- **RNF-PORT-003:** Las bases de datos deben ser compatibles con soluciones de código abierto o servicios gestionados en la nube que permitan la migración entre proveedores.

2.4. Seguridad

La seguridad abarca la protección del sistema y los datos contra accesos no autorizados, ataques y daños.

- **RNF-SEG-001:** Todas las comunicaciones externas e internas (entre microservicios) deben utilizar protocolos seguros (HTTPS/TLS).
- **RNF-SEG-002:** El sistema debe implementar autenticación basada en tokens (ej. JWT) para las APIs y OAuth2 para la autenticación de usuarios.
- **RNF-SEG-003:** Las contraseñas de los usuarios deben ser almacenadas mediante funciones hash criptográficas robustas y salting (ej. bcrypt).
- **RNF-SEG-004:** El sistema debe ser resistente a vulnerabilidades comunes como inyección SQL, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) y ataques de denegación de servicio (DoS).
- **RNF-SEG-005:** Se deben realizar auditorías de seguridad periódicas (ej. pruebas de penetración) al menos una vez al año.

2.5. Escalabilidad

La escalabilidad es la capacidad del sistema para manejar un aumento en la carga de trabajo o el volumen de datos de manera eficiente.

- **RNF-ESCAL-001:** Cada microservicio debe ser diseñado para escalar horizontalmente de forma independiente, permitiendo añadir o eliminar instancias según la demanda.
- **RNF-ESCAL-002:** El sistema debe soportar un crecimiento del 50% en el número de usuarios concurrentes y transacciones anuales sin una degradación perceptible del rendimiento.
- **RNF-ESCAL-003:** Las bases de datos deben ser capaces de escalar para manejar un crecimiento de datos del 30% anual.

2.6. Disponibilidad

La disponibilidad se refiere a la proporción de tiempo que el sistema está operativo y accesible para los usuarios.

- **RNF-DISP-001:** La plataforma debe tener una disponibilidad del 99.9% (equivalente a un tiempo de inactividad máximo de 8.76 horas al año).
- **RNF-DISP-002:** El sistema debe implementar mecanismos de tolerancia a fallos y recuperación automática para asegurar que la falla de un microservicio no cause una interrupción total del servicio.
- **RNF-DISP-003:** Se deben realizar copias de seguridad de los datos críticos diariamente, con un objetivo de tiempo de recuperación (RTO) máximo de 4 horas y un objetivo de punto de recuperación (RPO) máximo de 24 horas.

3. Restricciones Técnicas de la Arquitectura de Microservicios

La adopción de una arquitectura de microservicios introduce ciertas restricciones y desafíos técnicos que deben ser gestionados proactivamente.

3.1. Latencia de Red

- **Restricción-LAT-001:** La comunicación entre microservicios implica llamadas de red, lo que inherentemente introduce latencia. El diseño de las APIs y la granularidad de los servicios deben minimizar el número de llamadas inter-servicio para completar una operación de negocio.
- **Restricción-LAT-002:** Se debe considerar la implementación de patrones como API Gateway y la agregación de datos para reducir la latencia percibida por el cliente final.

3.2. Consistencia Eventual

- **Restricción-CONS-001:** En un sistema distribuido, la consistencia de los datos entre diferentes microservicios es a menudo eventual, no inmediata. Esto significa que los datos pueden no estar completamente sincronizados en todos los servicios en un momento dado.

- **Restricción-CONS-002:** El diseño de la lógica de negocio debe ser tolerante a la consistencia eventual, utilizando patrones como Sagas o Compensating Transactions para mantener la integridad de los datos a largo plazo.
- **Restricción-CONS-003:** Se deben comunicar claramente las implicaciones de la consistencia eventual a los stakeholders, especialmente en escenarios donde la inmediatez de los datos es crítica (ej. actualización de stock tras una compra).

3.3. Complejidad Operacional

- **Restricción-COMP-001:** La gestión de múltiples microservicios, cada uno con su propio ciclo de vida de desarrollo y despliegue, aumenta la complejidad operacional en comparación con una arquitectura monolítica.
- **Restricción-COMP-002:** Se requiere una infraestructura robusta para la orquestación de contenedores (ej. Kubernetes), la gestión de configuraciones, el descubrimiento de servicios y el balanceo de carga.
- **Restricción-COMP-003:** El equipo de operaciones debe tener experiencia en la gestión de sistemas distribuidos y herramientas de automatización (ej. CI/CD).

3.4. Gestión de Transacciones Distribuidas

- **Restricción-TRANS-001:** Las transacciones que abarcan múltiples microservicios son inherentemente complejas de gestionar, ya que el tradicional modelo ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) de las bases de datos relacionales no se aplica directamente.
- **Restricción-TRANS-002:** Se deben utilizar patrones de diseño específicos para transacciones distribuidas, como el patrón Saga, para asegurar la consistencia de los datos a través de los servicios.

3.5. Monitoreo y Observabilidad

- **Restricción-MON-001:** Es fundamental implementar un sistema de monitoreo y observabilidad integral que permita rastrear el rendimiento, la salud y el comportamiento de cada microservicio y las interacciones entre ellos.
- **Restricción-MON-002:** Se deben utilizar herramientas de agregación de logs (ej. ELK Stack, Grafana Loki), métricas (ej. Prometheus, Grafana) y trazabilidad

distribuida (ej. Jaeger, Zipkin) para diagnosticar problemas en un entorno de microservicios.

4. Conclusión

La comprensión y gestión proactiva de los requisitos no funcionales y las restricciones técnicas son pilares fundamentales para el éxito de la plataforma de e-commerce basada en microservicios. Al abordar el rendimiento, la mantenibilidad, la portabilidad, la seguridad, la escalabilidad y la disponibilidad desde las primeras etapas del diseño, y al reconocer y mitigar las complejidades inherentes a la arquitectura de microservicios, se puede construir un sistema robusto, eficiente y adaptable a las futuras demandas del mercado. Este documento sirve como una guía esencial para la toma de decisiones técnicas y la planificación estratégica del proyecto.

5. Referencias

- [Bass, L., Clements, P., & Kazman, R. \(2012\). *Software Architecture in Practice* \(3rd ed.\). Addison-Wesley.](#)
- [Newman, S. \(2015\). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.](#)
- [Vernon, V. \(2016\). *Implementing Domain-Driven Design*. Addison-Wesley.](#)
- [IEEE Standard for Software Requirements Specifications \(IEEE Std 830-1998\).](#)